Explorations in Decision Focused Bike Rebalancing

Spencer Loper
Carnegie Mellon University
sloper@cs.cmu.edu

Willem-Jan Van Hoeve
Carnegie Mellon University
vanhoeve@andrew.cmu.edu

1 Introduction

In the last five years, the U.S. has continued to invest in docked bike sharing systems, growing total docks by 40% to a total of $\approx 10,000$ [2]. As these systems continue to mature, efficient operations remains a critical challenge.

In docked bike sharing systems, stations with specific parking spots, or docks, are created for users to rent and return bikes. Importantly, each station has fixed capacity and must balance both the availability of bikes for outgoing rentals and the availability of docks for incoming returns. As such, operators consistently engage in rebalancing, where bikes are strategically moved between stations. Nonetheless, determining exactly how a system should be rebalanced is difficult and a rich area of research.

Central to bike rebalancing is the determination of each station's optimal, or target, level of inventory. Traditionally, this is done by using historical data to build a predictive model of each station's demand. Then, these predictions are used as an input to an optimization model to jointly solve for the stations' target inventories. While modern approaches have started leveraging advances in machine learning for better predictive modeling [13], many continue to rely on the Predict-then-Optimize (PTO) approach, treating prediction and optimization tasks independently.

However, recent work [1, 4] has enabled predictive models to be trained in-the-loop with downstream optimization tasks. This new approach, Decision-Focused-Learning (DFL), has shown promising results by better aligning machine learning models with downstream utility instead of upstream accuracy [3].

In this work, we explore the performance of the DFL approach in the context of bike rebalancing. Specifically, we solve for stations' target inventories by predicting net demand at each station and solving a mixed integer program (MIP). Importantly, we make use of the SPO+ surrogate loss to allow our predictive model to be trained end-to-end. We explore our approach's performance on historical bike sharing data from Pittsburgh (POGOH and Healthy Ride). These datasets are particularly interesting, since Pittsburgh's distinct seasonality and topology create meaningful asymmetries in demand patterns.

In short, our work finds add high-level findings

2 Related Work

Rebalancing in Bike Sharing Systems. Bike rebalancing has been widely studied and broadly consists of solving two problems: stations' target inventories and the subsequent vehicle routing when executing rebalancing. These problems are often modeled as MIPs [5] and can broadly be classified by when the rebalancing is occurring: static (overnight) or dynamic (intra-day). Given the reduced complexity from assuming overnight rebalancing, many papers focus on the static setting and, until recently, solved the two problems independently. However, recent work by Schuijbroek et al. [10] has demonstrated the value of solving for inventory targets and routes jointly. Nonetheless, these methods typically implement the PTO approach and treat demand forecasts as an independent

input to optimization problems. This two-staged approach allows for a potential disconnect between prediction accuracy and actual task performance.

Decision-Focused Learning. In order to directly align models with their downstream performance, significant work has been done to enable end-to-end learning. This approach, termed DFL, relies on numerous techniques for differentiating or approximating gradients through optimization problems [9]. A popular approach is to use the SPO+ surrogate loss function introduced by Elmachtoub and Grigas [4], which approximates downstream decision quality and can be used with binary and integer constraints. However, this approach is limited to formulations where the predictions are coefficients in a linear objective function. Another popular approach was first proposed by Amos and Kolter [1]. In this approach, differentiation through the KKT conditions allows for exact gradients. This approach, however, is limited to continuous formulations, though some work has demonstrated techniques for application in combinatorial settings [12].

Learning-Based Bike Rebalancing. Recent work has started exploring applications of machine learning in bike rebalancing. Most naturally are efforts to improve demand forecasting through more sophisticated modeling [6]. However, many have also started exploring reinforcement learning (RL) for solving bike rebalancing [8]. While these RL based approaches have shown promising results, they require the development of simulation environments and extensive computation to facilitate learning. Although DFL remains largely unexplored for bike rebalancing, it may offer a good balance of leveraging the known structure of the problem (through the optimization model) while still allowing task aware learning.

3 Methods

In this work, we implement the simplest bike rebalancing approach, static overnight rebalancing. Specifically, we assume that demand overnight is negligible and that our rebalancing process runs at midnight. Our process starts by predicting the net demand (outflow - inflow) at each station. Then, these net demands are passed to a MIP to solve for targets that balance the expected lost demand and over capacity events. To make demand predictions, we rely on cyclical embeddings of time and historical day-ahead weather forecasts. As a baseline this approach is implemented by training the predictive model independently with mean squared error (MSE). Then, a decision focused version is implemented using the SPO+ loss. We evaluate results by comparing objective values under our static rebalancing assumption and through a discrete event-based simulation. Details of each component are provided in the following sections.

3.1 Bike Sharing Dataset

We utilize historical bike sharing data from the city of Pittsburgh. As shown in fig. 1, Pittsburgh's bike sharing stations operated as Healthy Ride before being overhauled and renamed POGOH in early 2022. Due to this drastic change, we treat the two datasets as separate and implement independent experiments for both. Pittsburgh's bike sharing data provides a rich testing ground for algorithmic improvements, as the diversity in elevation and weather results in highly asymmetric demand patterns [7]. Notably, in late 2018, Healthy Ride more than doubled its stations from 50 to 105. Similarly, in July 2023, POGOH increased its station count from 42 to 60. Since these changes had significant impacts on the intra-station demand dynamics, we ignore all data present before these periods. Additionally, given the strong seasonality, we utilize an entire year for our test sets. So, for the Healthy Ride data we train with 2019-2020 and test on 2021. For POGOH, we train with August 2023-2024 and test on 2025.

3.2 Predictive Models & Features

In order to predict demand, we utilize a linear model $f : \mathbb{R}^7 \to \mathbb{R}^{|\mathcal{S}|}$, where $\mathcal{S}$ is the set of stations. These models use the features shown in table 1 and are trained with weight-decay. We did experiment with more complicated models (e.g. MLPs) but, given the relatively small amount of data, we struggled to achieve good generalization even with strong regularization.

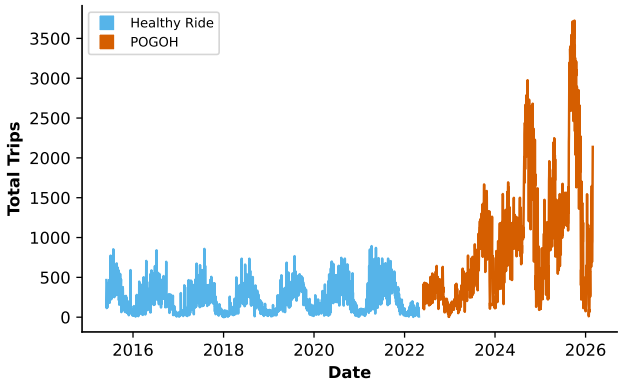

Figure 1: Daily total trips, showing Pittsburgh's strong seasonality and change from POGOH infrastructure improvements.

Table 1: Summary of features utilized in the predictive model for station-level demand forecasting.

Category	Feature Name	Description	Format
Temporal	Day of Week	Integer for the day of week (1,...,7)	Cyclical encoding
	Month of Year	Integer for the month of year (1,...,12)	Cyclical encoding
Weather	Temperature	Average feels like temperature (C)	Z-score
	Precipitation	Total of all precipitation (mm)	Log transform
	Wind	Average wind gust at 10m (km/h)	Z-score

3.3 Inventory Target Optimization Model

At the core of most bike rebalancing approaches is an optimization model. In our case, we utilize a MIP to minimize the costs of the predicted ending inventory positions subject to current capacity and inventory constraints. Our original formulation is not amenable to DFL, so we reformulate the problem in way that allows the use of the SPO+ surrogate loss function. Details are provided in the following sections.

3.3.1 Original Formulation

Let $\mathcal{S}$ be the set of stations and $C_i, I_i, \hat{d}_i$ be the capacity, current inventory, and estimated net demand for station i, respectively. Additionally, let c_l be the cost of lost demand and c_o the cost of over capacity and x_{ij} designate the amount of bikes to move from station i to j. Then, our original MIP is:

$$\min \quad \sum_{i \in \mathcal{S}} (c_l l_i + c_o o_i) \tag{1}$$

$$\text{s.t.} \quad I_i + \sum_{j \in \mathcal{S}} x_{ji} - \sum_{j \in \mathcal{S}} x_{ij} + l_i \geq \hat{d}i \qquad \forall i \in \mathcal{S} \tag{2}$$

$$I_i + \sum_{j \in \mathcal{S}} x_{ji} - \sum_{j \in \mathcal{S}} x_{ij} - \hat{d}_i - o_i \leq C_i \qquad \forall i \in \mathcal{S} \tag{3}$$

$$I_i + \sum_{j \in \mathcal{S}} x_{ji} - \sum_{j \in \mathcal{S}} x_{ij} \leq C_i \qquad \forall i \in \mathcal{S} \tag{4}$$

$$\sum_{j \in \mathcal{S}} x_{ij} \leq I_i \qquad \forall i \in \mathcal{S} \tag{5}$$

$$l_i, o_i \geq 0, \quad x_{ij} \in \mathbb{Z}_{\geq 0} \qquad \forall i, j \in \mathcal{S}$$

In this formulation, we use slack variables l_i and o_i to track the amount of lost demand and over capacity that is expected, given our predicted demand and rebalancing decision. Our objective, eq. (1),

is simply to minimize these total costs. In the constraints, eq. (2) calculates the lost demand, eq. (3) calculates the over capacity, eq. (4) ensures a station is not rebalanced to more than its capacity, and eq. (5) ensures a station does not send out more bikes than its current inventory. Notably, our focus is purely on the target inventories, so no cost is added for transporting bikes.

Unfortunately, this formulation does not allow for straightforward DFL implementation. Since the decision variables are integers, applying differentiable optimization techniques [1] is not possible. Moreover, because the predictions appear in the constraints and not in the objective, using the SPO+ loss function is also not possible [4]. Curiously, this formulation also has a feasible region that changes based on predictions and current inventories. However, the set of targets we can choose is always fixed between 0 and a station's capacity. We make use of this fact and re-formulate the target optimization model as described in the following section.

3.3.2 SPO+ Friendly Formulation

In the original formulation, eq. (2) and eq. (3) make applying DFL techniques difficult. However, these constraints only exist to calculate the expected costs from selecting a starting inventory level for a station. Since the feasible starting inventory levels for a station are always between 0 and its capacity, these costs can be pre-computed by enumerating all possible inventory levels. Specifically, let c_{ij} denote the cost of setting station i to inventory level j. Then, these costs can be pre-computed as:

$$c_{ij} = c_l \max(0, \hat{d}_i - j) + c_o \max(0, j - \hat{d}_i - C_i) \quad \forall j \in \{0, 1, ..., C_i\} \tag{6}$$

This operation, which we refer to as a cost head, can simply be repeated for each station to build a matrix of costs. Note, in practice all stations do not have the same capacity, so infeasible targets require masking to store in a matrix. After computing these costs, we let y_{ij} denote a binary decision variable for setting station i to level j. Then, the target optimization can be re-written as an integer program (IP):

$$\min \quad \sum_{i \in \mathcal{S}} \sum_{j=0}^{C_i} (c_{ij} y_{ij}) \tag{7}$$

$$\text{s.t.} \quad \sum_{j=0}^{C_i} y_{ij} = 1 \qquad \forall i \in \mathcal{S} \tag{8}$$

$$\sum_{i \in \mathcal{S}} \sum_{j=0}^{C_i} j \; y_{ij} = \mathcal{I} \tag{9}$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in \mathcal{S}, \forall j \in \{0, 1, ..., C_i\}$$

where $\mathcal{I}$ =total inventory $= \sum_{i \in \mathcal{S}} I_i$. Notably, this reformulation expands the number of variables from $|\mathcal{S}|$ to $|\mathcal{S}|(\max C_i + 1)$. However, in the case of our Pittsburgh bike sharing systems, this expansion is still a manageable few thousands variables.

Importantly, this reformulation has several, nice properties. First, the computation of the cost inputs in eq. (6) can be done with ReLUs, maintaining the differentiability of the predictive model. Secondly, the feasible region of this reformulation is constant and only depends on the total inventory in the system and the capacities of the stations. Finally, the objective is now a linear function of our predicted costs, enabling the use of SPO+ loss function. Using this reformulation, our end-to-end approach is visualized in fig. 2.

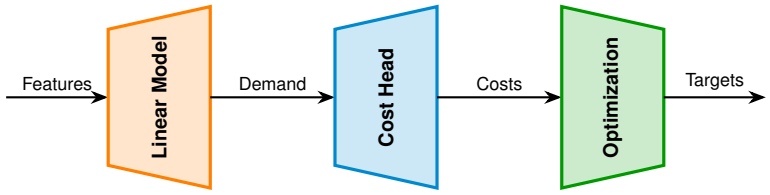

Figure 2: End-to-end architecture for decision-focused bike rebalancing.

3.3.3 SPO+ Routing Formulation

3.4 Simulation

In addition to evaluating the DFL approach under static rebalancing assumptions (the objective value of the optimization model), we also implement a discrete event-based simulation. Since the simulation is dynamic, it is more realistic and provides insights into the validity of the static rebalancing assumption.

At a high level, the simulation starts by executing rebalancing at midnight before stepping through historical trip data. As events occur, the simulation enforces stations' inventory constraints and tracks metrics. In the case of a historical trip attempting to rent when a station has no inventory, the trip is discarded and marked as lost demand. Similarly, if a historical trip tries to return to a station with no available docks, the trip is re-routed to the nearest station and marked as over capacity. If the nearest station also has no capacity, the bike is forced returned, temporarily allowing for a breach in capacity. This forced returning helps ensure the simulation runs quickly and mimics some unfortunate real-world behavior. Detailed pseudocode is provided in algorithm A1.

3.5 Implementation

Our implementation sources historical bike sharing data from the Western Pennsylvania Regional Data Center (WPRDC), augmented with historical day-ahead weather forecasts from open-meteo. The predictive models are implemented using PyTorch and the optimization models are implemented with Gurobi. To facilitate end-to-end training, we rely heavily on PyEPO [11]. Predictive models were trained using the Adam optimizer with ablations performed across learning rates and weight decay. Our implementation in its entirety can be found at `https://github.com/saloper/decision-focused-bike-rebalancing`.

3.6 Experiments

For each of our bike sharing datasets, we conducted the following experiments:

1. **Do Nothing:** As the name suggests, no rebalancing is done. This provides a lower bound for any acceptable rebalancing performance.
2. **Naive Capacity:** Bikes are rebalanced proportional to each station's capacity. In the case of fractional bikes, rounding is randomized between stations.
3. **Optimal:** Bikes are rebalanced using the approach in fig. 2, but ground truth demand predictions are used. This provides an upper bound on the performance of our static rebalancing approach.
4. **Linear MSE Loss:** Demand predictions are trained independently using MSE before executing the approach in fig. 2.
5. **Linear SPO+ Loss:** The full implementation of fig. 2, where demand predictions are trained end-to-end using SPO+.

For simplicity, all experiments were completed with $c_l = 1$ and $c_o = 1$ after some analysis revealed no meaningful change in behavior when these costs are treated unequally.

Notably, these experiments were completed using data from all hours of the day. However, many bike sharing systems face morning rushes followed by evening commutes. In our static rebalancing, these demands cancel each other out. So, we also completed these experiments focused on solving the morning rush with data only from midnight to noon. This variant will be referred to as the 12hr.

4 Results

5 Discussion

6 Future Work

- Datasets

- Scalability
- Robustness

References

[1] Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[2] Bureau of Transportation Statistics. Bikeshare and e-scooters in the u.s., 2026. Accessed: 2026-04-30.

[3] Priya Donti, Brandon Amos, and J. Zico Kolter. Task-based end-to-end model learning in stochastic optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] Adam N. Elmachtoub and Paul Grigas. Smart "predict, then optimize". *Management Science*, 2022.

[5] Sudipta Ghosh, Pradeep Varakantham, and Patrick Jaillet. Dynamic rebalancing in bike-sharing systems: Modeling, algorithms and applications. *European Journal of Operational Research*, 2024.

[6] Pierre Hulot, Daniel Aloise, and Sanjay Dominik Jena. Towards station-level demand prediction for effective rebalancing in bike-sharing systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018.

[7] Injung Kim, Konstantinos Pelechrinis, and Adam J. Lee. The anatomy of the daily usage of bike sharing systems: Elevation, distance and seasonality. In *Proceedings of the 22nd International Workshop on Urban Computing (UrbComp 2020)*. ACM, 2020.

[8] Jiaqi Liang, Defeng Liu, Sanjay Dominik Jena, Andrea Lodi, and Thibaut Vidal. Single- vs. dual-policy reinforcement learning for dynamic bike rebalancing, 2025.

[9] Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Victor Bucarey, Tias Guns, and Ferdinando Fioretto. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *Journal of Artificial Intelligence Research*, 80, 2024.

[10] J. Schuijbroek, R.C. Hampshire, and W.-J. van Hoeve. Inventory rebalancing and vehicle routing in bike sharing systems. *European Journal of Operational Research*, 257(3):992–1004, 2017.

[11] Bo Tang and Elias B Khalil. Pyepo: a pytorch-based end-to-end predict-then-optimize library for linear and integer programming. *Mathematical Programming Computation*, 2024.

[12] Bryan Wilder, Bistra Dilkina, and Milind Tambe. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. In *AAAI*, 2019.

[13] Yu Zhou, Qin Li, Xiaohang Yue, Jiajia Nie, and Qiang Guo. A novel predict-then-optimize method for sustainable bike-sharing management: a data-driven study in china. *Annals of Operations Research*, 359:5–37, 2022.

A Simulation Details

Algorithm A1 Discrete Event Based Bike Simulation

Require: Stations $s \in \mathcal{S}$ with capacities C_s, Daily event logs $\mathcal{E}_d$
Require: Demand predictor f_θ, Cost head g_ϕ, Optimization oracle $\mathcal{O}$
Ensure: Cumulative penalties: Lost Demand (L), Overcapacity (O), Forced Returns (F)

```
Initialize L ← 0, O ← 0, F ← 0
for each day d do
    % Phase 1: Predict-and-Optimize (Forward Pass)
    Predict demand: ŷ ← f_θ(x_d)
    Compute cost coefficients: c ← g_φ(ŷ)
    Solve for target inventories: w* ← O(c)
    Rebalance stations: I_s ← w*_s   ∀s ∈ S
    % Phase 2: Discrete-Event Simulation
    for each event (t, type, s) in E_d ordered by time do
        if type is rent then
            if I_s > 0 then
                I_s ← I_s − 1
            else
                L ← L + 1
                Mark this specific trip as failed
            end if
        else if type is return then
            if trip was marked failed then
                continue                          ▷ Cannot return a bike that was never rented
            end if
            if I_s < C_s then
                I_s ← I_s + 1
            else
                O ← O + 1                         ▷ Station full; incur penalty
                Let s′ be the nearest neighbor to s
                if I_s′ < C_s′ then
                    I_s′ ← I_s′ + 1
                else
                    F ← F + 1                     ▷ Neighbor also full; force return
                    I_s′ ← I_s′ + 1
                end if
            end if
        end if
    end for
end for
return L, O, F
```